

High-Level Synthesis Performance Prediction using GNNs: Benchmarking, Modeling, and Advancing

Nan Wu¹, Hang Yang³, Yuan Xie¹, Pan Li², and Cong Hao³

¹University of California - Santa Barbara, ²Purdue University, ³Georgia Institute of Technology
{nanwu, yuanxie}@ucsb.edu, panli@purdue.edu, {hyang628, callie.hao}@gatech.edu

ABSTRACT

Agile hardware development requires fast and accurate circuit quality evaluation from early design stages. Existing work of high-level synthesis (HLS) performance prediction usually requires extensive feature engineering after the synthesis process. To expedite circuit evaluation *from as early design stage as possible*, we propose rapid and accurate performance prediction methods, which exploit the representation power of graph neural networks (GNNs) by representing C/C++ programs as graphs. The contribution of this work is three-fold. **(1) Benchmarking.** We build a standard benchmark suite with 40k C programs, which includes synthetic programs and three sets of real-world HLS benchmarks. Each program is synthesized and implemented on FPGA to obtain post place-and-route performance metrics as the ground truth. **(2) Modeling.** We formally formulate the HLS performance prediction problem on graphs and propose multiple modeling strategies with GNNs that leverage different trade-offs between prediction timeliness (early/late prediction) and accuracy. **(3) Advancing.** We further propose a novel hierarchical GNN that does not sacrifice timeliness but largely improves prediction accuracy, significantly outperforming HLS tools. We apply extensive evaluations for both synthetic and *unseen* real-case programs; our proposed predictor largely outperforms HLS by up to 40× and excels existing predictors by 2× to 5× in terms of resource usage and timing prediction. The benchmark and explored GNN models are publicly available at <https://github.com/lydiawunan/HLS-Perf-Prediction-with-GNNs>.

1 INTRODUCTION

One essential requirement for hardware agile development is to evaluate circuit design quality quickly and accurately for rapid optimization iterations. Traditional electronic design automation (EDA) tools usually take hours to days as well as extensive manual efforts to accurately evaluate circuit quality. Although high-level synthesis (HLS) tools can greatly speed up circuit design, they still require minutes to hours for design synthesis before place and route, and the reported hardware performance metrics can be largely inaccurate [28]. Given the strong desire for hardware agile development and productivity boost, a quick and accurate performance evaluation at the earliest stage, even before HLS, is highly expected.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
DAC '22, July 10–14, 2022, San Francisco, CA, USA
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9142-9/22/07.
<https://doi.org/10.1145/3489517.3530408>

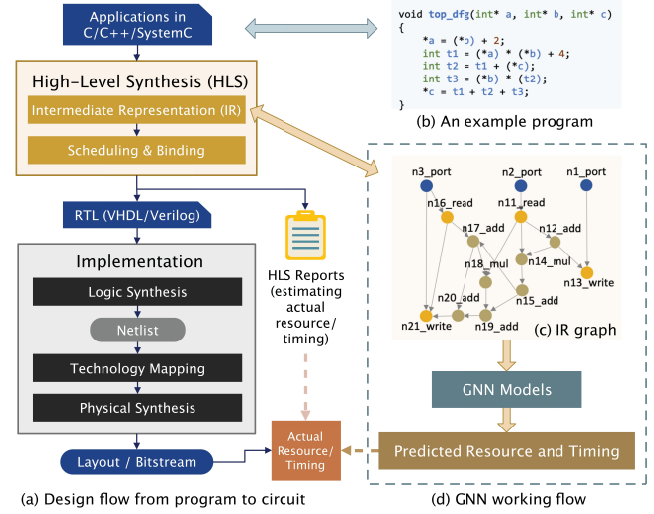


Figure 1: The overall performance prediction flow. (a) Design flow starting from behavioral programs to hardware circuits. (b) An example program written in C. (c) The intermediate representation (IR) graph extracted after front-end compilation. (d) The working flow of GNNs, predicting actual resource usage and timing merely based on raw IR graphs.

Studies in predicting hardware design quality of HLS tools usually focus on performance metrics before or after place and route, such as resource usage, timing, power, and area. Classic approaches use analytical models [20, 33, 34], which only work well for highly regular dataflows such as perfect loops and arrays. Recently, machine learning (ML)-based approaches have become promising in estimating the design performance [30]. Examples include Pyramid [16] that assembles several ML models to predict resource usage and timing, HLPredict [19] and XPPE [17] that leverage artificial neural networks (ANNs) to predict cross-platform performance.

Despite the great success, most ML-based methods require intensive feature extraction and empirical feature engineering from the hardware design flow. Figure 1(a) depicts the design flow from behavioral programs to circuits (e.g., layout/bitstream), including the HLS process and implementation. Notably, existing ML-based methods obtain copious features from HLS synthesis reports or intermediate results of a partially executed implementation process, which is time-consuming and demands extensive domain expertise.

Motivated by such limitations, in this work, we aim to approach HLS performance prediction *at its earliest stage with least feature engineering*, i.e., right after front-end compilation without invoking the core HLS synthesis process. Since compiled programs are often represented as intermediate representation (IR) graphs, we exploit the representation power of graph neural networks (GNNs) and

adopt various GNNs for timely performance prediction based on IR graphs. Figure 1(b)-(d) demonstrate our prediction flow: we extract IR graphs right after HLS front-end compilation and directly predict post-implementation circuit performance metrics.

To comprehensively investigate this problem, first, we propose prediction strategies at different stages of HLS and discuss their trade-offs between prediction accuracy and efficiency: the earlier the prediction is, the more beneficial it is for agile design but the less information can be acquired. Then, we propose a novel hierarchical GNN-based predictor that can *make predictions at the earliest stage but still utilize sufficient domain-specific information*. Further, to benefit the follow-up interdisciplinary research in EDA and GNN, we standardize the problem formulation and develop a rich benchmark suite. We summarize our contributions as follows:

- **Benchmarking.** We build a standard benchmark containing 40k synthetic C programs. Each program is compiled to derive the IR graph, synthesized by HLS tools, and implemented on FPGA to get the post-implementation performance metrics such as resource utilization and timing. Three sets of real-world benchmarks are also included for generalization evaluation.
- **Modeling.** To study trade-offs between prediction accuracy and efficiency, we propose two approaches directly using GNNs: (1) *off-the-shelf approach* at the earliest stage with the least domain-specific information; (2) *knowledge-rich approach* at a later stage with HLS auxiliary information to improve prediction accuracy.
- **Advancing.** We then propose a *knowledge-infused approach*, a novel and hierarchical GNN model that reaps timely prediction as well as ample domain knowledge, largely improving prediction accuracy with zero overhead at inference time. The prediction task is decoupled into a node-level classification task and a graph-level regression task, where the first classifies resource types and the second regresses numerical resource usage values.
- **Evaluation.** We apply extensive evaluations on both synthetic and *unseen* real-case programs; our proposed predictor largely outperforms the commercial Vitis HLS tool by up to 40× and excels existing predictor IRONMAN [28] by 2-5×, in terms of resource usage and timing prediction.
- **Open source for synergic research in HLS and GNN.** The benchmark and explored GNN models are publicly available at <https://github.com/lydiawunan/HLS-Perf-Prediction-with-GNNs>.

2 PERFORMANCE PREDICTION STRATEGIES

There are two fundamental questions for performance prediction: **when** and **how**.

When to predict. Circuit design quality can be predicted at different stages in the entire design flow, e.g., before or after HLS, or during implementation. For instance, HLS tools take in behavioral languages, convert them to RTL, and produce a synthesis report, which is an early estimation of the final performance, as shown in Figure 1. However, HLS prediction (synthesis reports) can be largely inaccurate [24, 28] even after minutes or hours of synthesis. Among existing ML-based predictors, both Pyramid [16] and XPPE [17] estimate resource usage and timing after HLS by extracting features from HLS synthesis reports, while HLSPredict [19] predicts FPGA cycle count and power before HLS. Typically, an

early and timely prediction is beneficial for agile development, but little useful domain-specific knowledge is exposed at this stage.

How to predict. Existing ML-based prediction approaches attempt linear regression, ANN, support vector machine, random forest, and ensemble models. Although promising, these models require heavy feature engineering to provide sufficient features as model inputs. For instance, Pyramid [16]/XPPE [17] and HLSPredict [19] require up to 183 and 75 features, respectively, which can only be obtained by *actually running HLS or CPU/FPGA sub-trace generation*. Given unseen designs, they cannot be directly applied without re-running HLS or implementation, and thus do not *generalize* well.

Our prediction strategy. Our goal is to assist agile hardware development by making the performance prediction *as early as possible* and *as accurate as possible*. Specifically, we focus on predicting the *implemented*, i.e., post place-and-route, design performance metrics on FPGA, including resource utilization and critical path (CP) timing, without invoking HLS or implementation processes.

To address existing limitations (i.e., late prediction and weak generalizability), we propose three early prediction approaches (as illustrated in Figure 2). To make it *timely*, we perform prediction based on the IR graph of a program, i.e., data flow graph (DFG) and control data flow graph (CDFG), which can be quickly extracted after the front-end compilation [1] within seconds. To make it *generalizable*, we propose to apply GNNs on DFGs/CDFGs, exploiting the inductiveness of GNNs to make accurate predictions for completely unseen designs without retraining. Specifically, the three approaches are:

- *Off-the-shelf approach.* The first approach directly predicts design performance metrics at the earliest stage by taking IR graphs (DFG/CDFG) as inputs to GNN models. The features are extracted right after HLS front-end compilation, leading to the fastest prediction since the compilation usually only takes a few seconds.
- *Knowledge-rich approach.* The second approach draws support from auxiliary domain information distilled from intermediate HLS results: the resource usage associated with each node. The features must be obtained during HLS execution, resulting in late (after partially executing the HLS process but no implementation) but more accurate predictions.
- *Knowledge-infused approach.* The third approach is a *hierarchical GNN-based* prediction strategy that reaps advantages of the previous two approaches: not only does it make the earliest prediction but also benefits from domain knowledge with almost zero overhead during inference. The knowledge infusion is achieved by decoupling the prediction task into two steps: the first step is node-level classification for resource types, in which the domain knowledge is infused during the training phase; the second step is graph-level regression that estimates the numerical resource usage and timing on top of the self-inferred domain knowledge.

Details of the three proposed approaches are presented in Section 4. Prior to that, we introduce our benchmark suite in Section 3.

3 BENCHMARKING

To facilitate more interdisciplinary research in ML and EDA, we build a standard benchmark to promote rapid performance prediction, which includes plenty of synthesizable programs together with *actual* (i.e., post-implementation) performance metrics on FPGA.

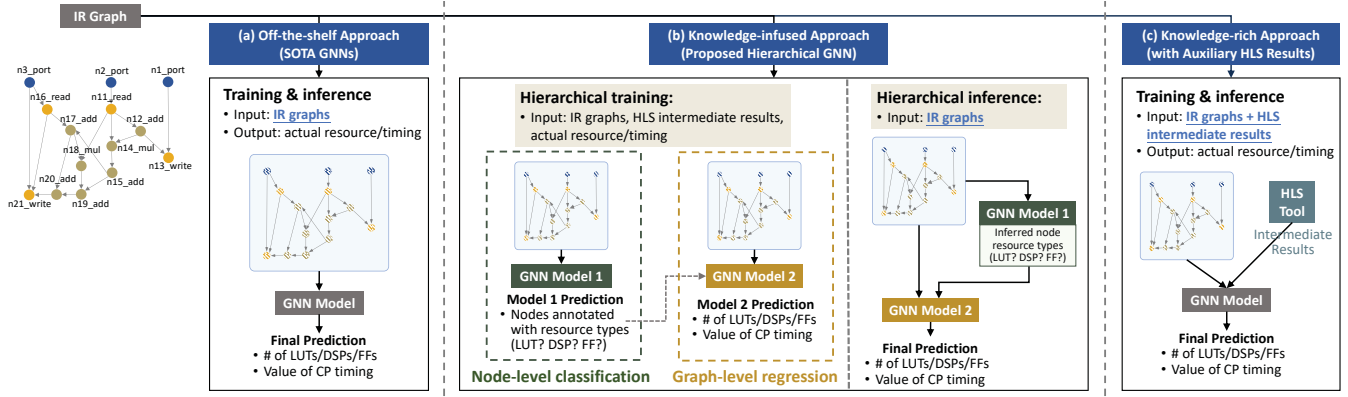


Figure 2: Our three proposed approaches: (a) off-the-shelf approach at the earliest stage for prediction; (b) knowledge-infused approach, also at the earliest stage but with self-inferred domain-specific information; (c) knowledge-rich approach after partial execution of HLS to obtain auxiliary information.

3.1 Benchmark Format

Input. The input to a predictor is the IR graph of a program, which can be quickly extracted after the front-end compilation [1]. In HLS, DFGs and CDFGs are the most common IR graphs. Specifically, DFGs are extracted from *basic blocks*, a straight-line code sequence with no branches in except to the entry and no branches out except at the exit [11], and thus they are directed acyclic graphs without any structural loops; CDFGs are extracted from programs with loops and branches, and thus they contain additional nodes and edges/loops for control dependency.

Node/Edge Features. The three proposed approaches use different sets of node features, as listed in Table 1. After HLS front-end compilation, there are seven node features immediately available for the off-the-shelf approach, such as node category, bitwidth, and opcode. For the knowledge-infused and the knowledge-rich approaches, we additionally include the resource type and the number of used resources in node features, respectively. Notably, for the knowledge-infused approach, the auxiliary node features (i.e., resource type) are *only* provided during training, while during inference they are inferred by our node-level GNN. Each edge has two features, the edge type represented in integers, and a binary value indicating whether this edge is a back edge.

Tasks/labels. We provide two types of tasks, a *node-level classification* task, and a *graph-level regression* task, where the former is easier than the later.

- For the *node-level classification* task, each node in IR graphs is assigned a label indicating the resource type(s) that the node will use in its final implementation. We consider three resource categories: DSP, LUT, and FF. A node can be implemented by zero, one, or multiple resource types. For example, a `sdiv` node may use both DSP and LUT; a `partselect` node only uses FF; a control-related node may use nothing, e.g., `br` that indicates a branch entry. We organize the resource type prediction as three binary classification tasks.
- For the *graph-level regression* task, each graph is labeled based on its post-implementation performance metrics. We consider four metrics for regression: DSP, FF, LUT, and CP. The first three

Table 1: Node features and example values.

Feature	Description	Values
Off-the-shelf approach with minimum domain information		
Node type	General node type	operation nodes, blocks, ports, misc
Bitwidth	Bitwidth of the node	0~256, misc
Opcode type	Opcode categories based on LLVM	binary_unary, bitwise, memory, etc.
Opcode	Opcode of the node	load, add, xor, icmp, etc.
Is start of path	Whether the node is the starting node of a path	0, 1, misc
Cluster group	Cluster number of the node	-1~256, misc
Knowledge-infused and knowledge-rich approach		
DSP	DSP used for this node?	binary/integer values, misc
LUT	LUT used for this node?	binary/integer values, misc
FF	FF used for this node?	binary/integer values, misc

are integer numbers indicating the number of resources used in final implementation; the last one is CP timing slack in fractional number, determining the maximum working frequency of FPGA.

3.2 Benchmark Generation

We construct the synthesizable benchmark suite including synthetic C programs as well as real-world HLS applications. The synthetic programs fall into two categories, basic blocks that derive DFGs, and programs with control loops and branches that derive CDFGs. All of the synthetic programs are generated by the C program generator `ldrgen` [2], a variant of `Frama-C` [13]. For graph-level tasks, there are 19,120 and 18,570 C programs in the DFG and CDFG datasets, respectively. The node-level dataset contains more than 660k nodes derived from DFGs and CDFGs. In addition, there are three sets of real-world HLS applications: `MachSuite` [22], `CHStone` [10], and `PolyBench/C` [21], consisting 16, 10, and 30 different applications, respectively. The real-world applications are *only* used for generalization evaluation of GNN models and we do not include them during training. More statistics of the benchmark suite is analyzed in Program-to-Circuit [29].

4 MODELING AND ADVANCING WITH GNNS

This section details the three proposed GNN-based approaches with various trade-offs between timeliness and accuracy. GNNS operate by propagating information along the edges of a given graph. By stacking multiple GNN layers, each node can receive information from multi-hop neighbors and locally characterize the corresponding receptive field for node-level tasks. Graph pooling then summarizes global information to perform graph-level prediction tasks.

4.1 Modeling: Off-the-Shelf Approach with State-of-the-Art GNN Models

In the off-the-shelf approach, we screen several state-of-the-art GNN models, aiming to identify (1) which properties of existing GNN models would help with resource/timing prediction and (2) how domain-specific insights can be combined with these properties to improve prediction accuracy. 14 different GNN models are selected from four categories based on how topological and relational information in graphs are exploited. We briefly introduce them as follows.

- **Graph convolutional network (GCN) and variants:** (1) GCN [12]; (2) GCN equipped with a virtual node [8]; (3) SGC [27], a simplified version of GCN; (4) GraphSage [9], a variant of GCN sampling a fixed number of neighboring nodes to keep the computational footprint consistent; (5) ARMA [3], a variant of GCN with auto-regressive moving average filters; (6) PAN [15], a path-integral-based GCN variant assigning trainable weights to each path based on its length. Previous work IRONMAN [28] also has a similar GCN-based performance predictor.
- **Graph isomorphism network (GIN) and variants:** (1) GIN [32], provably as powerful as the Weisfeiler-Lehman graph isomorphism test for discrete input feature space; (2) GIN with a virtual node [8]; (3) PNA [5], leveraging complementary aggregators to better understand graph structures and retain neighborhood information, especially for continuous input feature space.
- **Employing multi-relational information:** (1) GAT [25], using attention mechanisms to implicitly assign different importance to nodes in the same neighborhood; (2) GGNN [14], using trainable edge-dependent weights with gated recurrent units; (3) RGCN [23], using trainable edge-dependent weights with non-linear activations.
- **Inspired from vision tasks:** (1) Graph U-Net [7], using an encoder-decoder structure on graphs; (2) GNN-FiLM [4], combining feature-wise linear modulation (FiLM) with the message passing procedure.

To fairly evaluate these models, we use the same GNN structure (e.g., same embedding size, same layer count) but with different types of GNN layers. The goal is to directly predict actual resource/timing based on IR graphs without invoking HLS. This approach makes the earliest prediction since HLS front-end compilation is the very first step of an EDA design flow. While with the best timeliness, the accuracy is compromised due to the ignorance of hardware-specific information. For example, the GNN model does not have the information of what resources will be used to implement a certain node (e.g., LUT or DSP?), making it hard to accurately predict the total resource utilization of entire IR graphs.

4.2 Modeling: Knowledge-Rich Approach with Selected GNN Models

To incorporate more hardware-specific information revealed during the synthesis flow, we devise the knowledge-rich approach, which takes both IR graphs and auxiliary information from intermediate HLS results as inputs, as shown in Figure 2(c). The auxiliary information from HLS tools indicates both the resource type(s) and the number of each type of resource used in final implementation for every node in IR graphs. As each node is marked with pre-characterized resource estimations, the GNN model can pay more attention to resource interference/sharing among nodes, achieving much better prediction accuracy.

Armed with rich domain knowledge, this approach emphasizes more on prediction accuracy, especially for resource estimation, yet compromises timeliness and efficiency since HLS tools do take some time to generate intermediate results. Evaluation of this approach is conducted with the top performant GNN models in the screening from the off-the-shelf approach.

4.3 Advancing: Knowledge-Infused Approach with Hierarchical GNN Models

To strike a balance between timeliness and accuracy, we propose the knowledge-infused approach with hierarchical GNN models. As depicted in Figure 2(b), the resource/timing prediction is disentangled into two tasks: a node-level classification task that annotates resource types associated with each node, and a graph-level regression task that predicts actual resource/timing with the annotated graphs. We use two separate GNN models for these two tasks, and adopt hierarchical training and inference.

- **Hierarchical training.** First, the node-level classification GNN takes IR graphs as inputs, and the domain knowledge is infused by providing labels to each node that denote resource types used in final implementation based on HLS intermediate results. Second, the graph-level regression GNN then takes both IR graphs and ground-truth resource types as inputs, aiming to convey the infused domain knowledge from node-level to graph-level tasks and to improve final prediction accuracy.
- **Hierarchical inference.** During inference, the only inputs required for the two trained GNNs are the IR graphs. First, the node-level GNN model infers resource types for each node. Second, combining the node-level inference results with original IR graphs, the graph-level regression GNN grasps self-inferred domain knowledge to perform final predictions.

Taking advantages of knowledge infusion during training, this approach demonstrates a great balance between timeliness and accuracy: predicting resource/timing from the earliest stage and simultaneously adopting adequate domain information to improve prediction accuracy.

5 EXPERIMENT

5.1 Experimental Setup

All GNN models are implemented with Pytorch Geometric [6]. The ground-truth (actual) resource usage (LUT/DSP/FF) and CP timing are synthesized by Vitis HLS [26] and implemented by Vivado [31]. DFG and CDFG datasets are randomly split into 80% train, 10%

validation and 10% test; real-world benchmarks are only used for generalization evaluation. Each GNN is empirically set as five layers with a hidden-dimension size of 300. For graph-level regression, sum or mean pooling is used to derive graph representations, followed by a feed-forward network with the structure 300-600-300-1. Models are trained using Adam optimizer for 100 epochs. Learning rates, dropout and other hyper-parameters are tuned on the validation set. Each model is trained with five runs using different random number seeds and we report the average of three with least validation error.

5.2 Modeling: SOTA GNN Analysis

We launch discussions of the off-the-shelf approach from three aspects: (1) how different applications (i.e., graph structures) influence prediction accuracy; (2) which properties of existing GNN models would help improve accuracy; (3) what domain-specific insights can be derived to facilitate future graph representation learning on fast and generalizable evaluation in EDA tasks.

Different graphs: DFG vs. CDFG. Table 2 exhibits mean absolute percentage error (MAPE) of predictions on DFGs and CDFGs from synthetic programs. The MAPE on CDFGs is larger than that on DFGs, which attributes to two major reasons. First, DFGs do not have loops but CDFGs typically include a considerable number of loops [29]. Since message-passing-based GNN models have limited expressiveness and are not better than the 1-Weisfeiler-Lehman isomorphism test [18], they are not excelled to handle graphs with many loops. Second, control signals introduce additional nodes/edges that represent control states and dependency, which are seemingly unrelated to resource usage. These nodes/edges easily confuse GNN models during resource prediction.

GNN model analysis. PNA and RGCN generally show superior performance, implying two takeaways. First, the relational information (i.e., edge information) is important in IR graphs, since it represents data or control dependency, or a mix of both, which is a critical basis in logic synthesis and impacts resource allocation. Second, equipped with multiple aggregators, PNA is more powerful to characterize different neighborhood information, thus providing better prediction accuracy.

Domain-specific insights. (1) *Resource.* Among three types of resource, DSPs are mainly used for computation; FFs often relate to memory operations and small arrays; LUTs may appear in computation, memory or control nodes. The key to making precise DSP prediction is to distinguish major computation nodes that are most likely to use DSPs. For instance, a multiplication node with a large bitwidth tends to use DSPs, while divisions and bitwise operations prefer LUTs. Similarly, effective extraction of memory-related nodes would greatly benefit FF predictions. Since LUTs are involved in the entire graph (as computation units and glue logic to circuit components), graph-level understanding is important. To briefly summarize, it is helpful to carefully characterize neighborhood information from each node’s predecessors, successors, itself, and their relations, such that the sophisticated mapping rules from heterogeneous nodes to resource usage can be clearly understood and quantitatively learned. (2) *Timing.* Compared with resource predictions, CP timing predictions show relatively lower MAPE and better consistency between DFGs and CDFGs. A probable reason is that CP timing is local information and thus is insensitive to graph sizes as long as the critical path segment can be recognized.

Table 2: MAPE of graph-level regression with different GNN models on DFG and CDFG datasets. The top two performant models are marked in bold.

	DFG				CDFG			
	DSP	LUT	FF	CP	DSP	LUT	FF	CP
GCN	16.31%	16.49%	21.27%	6.12%	25.30%	28.64%	38.34%	8.79%
GCN-V	15.72%	15.93%	21.64%	6.36%	17.31%	33.93%	39.94%	8.13%
SGC	42.12%	23.93%	30.61%	7.92%	44.01%	60.87%	53.50%	10.32%
SAGE	15.18%	14.01%	17.11%	6.12%	17.01%	28.09%	39.11%	8.25%
ARMA	19.12%	13.46%	16.87%	6.50%	18.47%	25.21%	32.15%	8.42%
PAN	15.24%	14.13%	17.23%	6.38%	16.88%	32.65%	44.36%	8.54%
GIN	15.52%	16.10%	22.08%	6.58%	15.47%	28.48%	38.82%	8.76%
GIN-V	15.04%	16.17%	23.09%	6.40%	17.94%	29.40%	48.64%	8.59%
PNA	12.65%	11.64%	14.41%	6.26%	14.71%	22.86%	26.47%	8.87%
GAT	26.22%	22.64%	27.74%	8.30%	28.66%	46.19%	54.73%	10.32%
GGNN	15.40%	13.64%	16.94%	6.47%	16.28%	28.05%	31.88%	8.50%
RGCN	13.27%	13.03%	15.09%	6.14%	15.03%	26.33%	25.52%	8.72%
UNet	18.40%	14.90%	19.17%	6.61%	18.92%	32.83%	53.06%	9.02%
FiLM	20.05%	12.50%	16.94%	6.27%	17.42%	26.97%	27.35%	8.67%

5.3 Advancing: Comparison of Three Approaches

We first discuss the results of the knowledge-infused approach, and then comprehensively compare the three proposed approaches with commercial HLS and the prior work IRONMAN [28].

Knowledge-infused approach. Essentially, using GNNs to predict actual resource/timing from IR graphs is to approximate the set of sophisticated heuristics and mapping rules used by HLS scheduling/binding and logic/physical synthesis during design flow. The evaluation of the off-the-shelf approach in Table 2 indicates that *plug-in application of GNNs cannot well approximate such underlying rules*. Thus, in addition to infusing domain knowledge during training, another motivation of the hierarchical structure in the knowledge infused approach is to *divide and conquer*. The complicated performance prediction is decoupled as two simpler tasks that are solved separately. For *node-level classification*, Table 3 shows prediction accuracy of classifying resource types, where high accuracy is achieved for most of the cases since local neighborhood characterization is enough for node-level resource type classification. For *graph-level regression*, Table 4 displays MAPE of predictions on synthetic programs, showing an accuracy boost compared with the off-the-shelf approach.

With the hierarchical training, both the node-level and the graph-level GNN models in Figure 2(b) are approximating simplified design heuristics. Specifically, the node-level classification aims to understand the preference of resource types on different nodes; the graph-level regression focuses on globally estimating resource sharing and interference among nodes. With the hierarchical inference, the domain knowledge infused during training can be self-inferred when encountering unseen designs, leading to improved prediction accuracy from the earliest design stage.

Accurate, timely, and generalizable. The three approaches explore different trade-offs between timeliness and accuracy. Intuitively, the more domain information is leveraged, the more accurate predictions are provided, whereas the longer time would be taken for feature collection. The off-the-shelf approach makes predictions from the earliest stage simply with IR graphs, at the cost of accuracy loss due to ignorance of domain knowledge. The knowledge-rich

Table 3: Prediction accuracy of node-level resource classification with four different GNN models on DFGs, CDFGs and real-case applications.

	DFG			CDFG			Real Case		
	DSP	LUT	FF	DSP	LUT	FF	DSP	LUT	FF
GCN	93.79%	84.84%	88.66%	83.00%	77.01%	64.74%	79.70%	81.83%	86.82%
SAGE	93.06%	87.32%	92.09%	85.65%	78.41%	60.40%	87.39%	86.44%	55.88%
GIN	93.80%	84.93%	91.57%	79.24%	73.05%	65.78%	74.70%	75.53%	72.24%
RGCN	93.91%	87.13%	91.52%	85.80%	78.46%	68.92%	90.82%	88.83%	91.55%

approach provides the best prediction accuracy, yet has to wait for HLS tools providing intermediate results, sacrificing timeliness. The knowledge-infused approach shows a balance: infusing adequate domain knowledge during training, and making predictions from the earliest stage during inference.

Generalization capability is a key indicator of whether an ML- or GNN-based approach can be widely applied for certain EDA tasks. Table 5 shows MAPE of the three proposed approaches and Vitis HLS on real-case applications. Compared with Vitis HLS, our approaches significantly improve prediction accuracy especially for LUT/FF usage and CP timing. Specifically, PNA-based knowledge-infused approach outperforms HLS by 1.2× to 40.6×, and PNA-based knowledge-rich approach outperforms HLS by 1.7× to 51.4×. Note that since IRONMAN [28] is a variant of the off-the-shelf GCN, whose performance is inferior to RGCN, the results in Table 5 imply that our proposed hierarchical GNN-based strategy outperforms IRONMAN by at least 2.1× to 5.0×.

Such results empirically demonstrate (1) generalization capability not only from seen to unseen designs but also from synthetic to realistic applications, and (2) accuracy and timeliness conspicuously surpassing HLS tools.

6 CONCLUSION

In this work, we discuss three approaches for early circuit performance prediction using GNNs: (1) the off-the-shelf approach, which makes the earliest prediction with least domain-specific information, showing on-par performance with HLS; (2) the knowledge-rich approach, which makes late prediction after HLS with auxiliary information, showing significantly better performance than HLS; (3) the knowledge-infused approach, which makes the earliest prediction in a two-step hierarchical manner with self-inferred knowledge, still significantly outperforming HLS. We also construct a standard benchmark suite to facilitate future research. This work not only demonstrates the great potential of applying GNNs in EDA, but also advances the GNN design by proposing innovative architectures.

REFERENCES

- [1] Alfred V Aho et al. 2007. *Compilers: Principles, Techniques and Tools*. Addison wesley (2007).
- [2] Gergő Barany. 2017. Liveness-driven random program generation. In *International Symposium on Logic-Based Program Synthesis and Transformation*. Springer.
- [3] Filippo Maria Bianchi et al. 2021. Graph neural networks with convolutional arma filters. *IEEE TPAMI* (2021).
- [4] Marc Brockschmidt. 2020. Gnn-film: Graph neural networks with feature-wise linear modulation. In *2020 ICML*.
- [5] Gabriele Corso et al. 2020. Principal neighbourhood aggregation for graph nets. *2020 NeurIPS* (2020).
- [6] Matthias Fey and Jan Eric Lenssen. 2019. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428* (2019).
- [7] Hongyang Gao and Shuiwang Ji. 2019. Graph u-nets. In *2019 ICML*.
- [8] Justin Gilmer et al. 2017. Neural message passing for quantum chemistry. In *International conference on machine learning*.

Table 4: MAPE of the three proposed approaches with RGCN/PNA on DFG and CDFG datasets. The default notation means the off-the-shelf approach; -I means the knowledge-infused approach; -R means the knowledge-rich approach.

	DFG				CDFG			
	DSP	LUT	FF	CP	DSP	LUT	FF	CP
RGCN ¹	13.27%	13.03%	15.09%	6.14%	15.03%	26.33%	25.52%	8.72%
RGCN-I	10.60%	10.25%	12.47%	5.70%	12.65%	20.55%	19.01%	6.78%
RGCN-R	8.86%	8.58%	10.18%	4.91%	10.98%	14.06%	16.65%	5.46%
PNA	12.65%	11.64%	14.41%	6.26%	14.71%	22.86%	26.47%	8.87%
PNA-I	8.26%	5.10%	7.58%	5.51%	10.39%	14.12%	16.42%	6.54%
PNA-R	7.06%	4.02%	5.78%	5.39%	8.95%	10.27%	11.22%	5.81%

Table 5: Testing MAPE of the three proposed approaches with RGCN/PNA on real-case applications.

	HLS	RGCN	RGCN-I	RGCN-R	PNA	PNA-I	PNA-R
DSP	26.07%	45.61%	40.89%	32.90%	40.06%	21.95%	15.20%
LUT	871.56%	66.23%	30.91%	24.08%	56.34%	21.45%	16.96%
FF	322.86%	101.20%	38.75%	27.72%	47.65%	20.10%	17.42%
CP	32.09%	8.13%	5.35%	5.83%	8.68%	4.80%	3.97%

- [9] Will Hamilton et al. 2017. Inductive representation learning on large graphs. *2017 NeurIPS* (2017).
- [10] Yuko Hara et al. 2009. Proposal and quantitative analysis of the CHStone benchmark program suite for practical C-based high-level synthesis. *JIP* (2009).
- [11] John L Hennessy and David A Patterson. 2011. *Computer architecture: a quantitative approach*. Elsevier.
- [12] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [13] Florent Kirchner et al. 2015. Frama-C: A software analysis perspective. *Formal Aspects of Computing* 27, 3 (2015), 573–609.
- [14] Yujia Li et al. 2016. Gated Graph Sequence Neural Networks. In *2016 ICLR*.
- [15] Zheng Ma et al. 2020. Path integral based convolution and pooling for graph neural networks. *2020 NeurIPS* (2020).
- [16] Hosein Mohammadi Makrani et al. 2019. Pyramid: Machine learning framework to estimate the optimal timing and resource usage of a high-level synthesis design. In *2019 FPL*. IEEE.
- [17] Hosein Mohammadi Makrani et al. 2019. XPPE: cross-platform performance estimation of hardware accelerators using machine learning. In *2019 ASP-DAC*.
- [18] Haggai Maron et al. 2019. Provably powerful graph networks. *2019 NeurIPS* (2019).
- [19] Kenneth O’Neal et al. 2018. Hlspredict: Cross platform performance prediction for fpga high-level synthesis. In *2018 ICCAD*. IEEE.
- [20] André Bannwart Perina et al. 2019. Lina: Timing-constrained high-level synthesis performance estimator for fast DSE. In *2019 ICFPT*. IEEE.
- [21] Louis-Noël Pouchet and Tomofumi Yuki. 2016. Polyhedral Benchmark suite. <http://web.cs.ucla.edu/~pouchet/software/polybench/>.
- [22] Brandon Reagen et al. 2014. Machsuite: Benchmarks for accelerator design and customized architectures. In *2014 IISWC*.
- [23] Michael Schlichtkrull et al. 2018. Modeling relational data with graph convolutional networks. In *2018 ESWC*. Springer.
- [24] Ecenur Ustun et al. 2020. Accurate operation delay prediction for FPGA HLS using graph neural networks. In *2020 ICCAD*.
- [25] Petar Veličković et al. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [26] Vitis. Accessed: 2022. *Vitis High-Level Synthesis User Guide (UG1399)*. <https://docs.xilinx.com/r/en-US/ug1399-vitis-hls>.
- [27] Felix Wu et al. 2019. Simplifying graph convolutional networks. In *2019 ICML*.
- [28] Nan Wu et al. 2021. Ironman: Gnn-assisted design space exploration in high-level synthesis via reinforcement learning. In *2021 GLSVLSI*.
- [29] Nan Wu et al. 2021. Program-to-Circuit: Exploiting GNNs for Program Representation and Circuit Translation. *arXiv preprint arXiv:2109.06265* (2021).
- [30] Nan Wu and Yuan Xie. 2022. A survey of machine learning for computer architecture and systems. *ACM Computing Surveys (CSUR)* 55, 3 (2022), 1–39.
- [31] Xilinx. Accessed: 2022. *Xilinx Vivado*. <https://www.xilinx.com/products/design-tools/vivado.html>.
- [32] Keyulu Xu et al. 2018. How Powerful are Graph Neural Networks?. In *2018 ICLR*.
- [33] Jieru Zhao et al. 2017. COMBA: A comprehensive model-based analysis framework for high level synthesis of real applications. In *2017 ICCAD*. IEEE.
- [34] Jieru Zhao et al. 2019. Performance modeling and directives optimization for high-level synthesis on FPGA. *IEEE TCAD* (2019).

¹An advanced version of IRONMAN [28] by considering relational information.